

UIE : Intelligence Artificielle, Licence 2

Travaux Pratiques : Analyse de données avec Python

Service Web pour la Data Science
Solemane COULIBALY

(solemane.coulibaly@gmail.com) avril 2026

Table des matières

1 Objectifs pédagogiques	2
2 Bonnes pratiques pour une API d'analyse de données	2
3 Mini-projets pour ingénieurs IA	2
3.1 Exercice 1 : Service de classification d'iris en temps réel	2
3.2 Exercice 2 : Analyse statistique descriptive à la demande	3
4 Indications supplémentaires	4
4.1 Documentation de l'API	4
4.2 Gestion des erreurs	4
4.3 Tests d'intégration	4
5 Rapport et Livrables	4
5.1 Livrables attendus	4
5.2 Contenu du rapport	4
6 Critères d'Évaluation	5
7 Fichier requirements.txt minimal	5

1 Objectifs pédagogiques

- Comprendre comment exposer des traitements d'analyse de données et de machine learning via une API REST.

- Utiliser Flask pour créer des endpoints recevant des données et retournant des résultats analytiques.
- Manipuler les bibliothèques numpy, matplotlib et scikit-learn dans un contexte de service web.
- Concevoir des API robustes, documentées et adaptées à des projets d'IA.

2 Bonnes pratiques pour une API d'analyse de données

- Gestion des erreurs (try/except, codes HTTP appropriés)
- Sérialisation des types numpy (utiliser tolist())
- Limitation de la taille des requêtes
- Documentation interactive avec Swagger/Flask-RESTX (optionnel) — Déploiement simple avec gunicorn

3 Mini-projets pour ingénieurs IA

Chaque projet inclut :

- **Prérequis techniques** : Flask, numpy, matplotlib, scikit-learn — **Livrable attendu** : rapport, code source, fichier requirements.txt, exemple de requête client (cURL ou script Python)

3.1 Exercice 1 : Service de classification d'iris en temps réel

Contexte Déployer un modèle de classification entraîné sur le dataset Iris pour que des applications externes puissent prédire l'espèce d'une fleur à partir de mesures saisies.

Objectifs

- Fournir un modèle LogisticRegression pré-entraîné et sauvegardé avec joblib
- Exposer un endpoint POST /predict recevant un JSON contenant sepal_length, sepal_width, petal_length, petal_width
- Retourner la classe prédite et les probabilités associées
- Ajouter un endpoint GET /model/info renvoyant les métadonnées du modèle (accuracy, features importances, classes)
- Ajouter un endpoint GET /plot/residuals : génère un graphique des résidus du modèle (à l'aide de matplotlib) et le renvoie sous forme d'image PNG.

Étapes

1. Entraînement du modèle en local (script séparé) et sauvegarde
2. Chargement du modèle au démarrage de l'API Flask
3. Validation des entrées (valeurs numériques, plages acceptables)

4. Formatage de la réponse JSON incluant predicted_class (nom de l'espèce) et probabilités

5. Gestion des erreurs (ex : données manquantes)

Exemple de réponse

```
{
  "prediction": " setosa ",
  "probabilities": {
    " setosa ": 0.98 ,
    " versicolor ": 0.02 ,
    " virginica ": 0.00
  }
}
```

GET /model/info

```
{
  "model_type": " LogisticRegression ",
  "accuracy": 0.78 ,
  "features": [ " sepal_length ", " sepal_width ", " petal_length ", " petal_width " ],
  "feature_importances": {
    " sepal_length ": 0.32 ,
    " sepal_width ": 0.21 ,
    " petal_length ": 0.47 ,
    " petal_width ": 0.00
  }
}
```

Codes HTTP attendus

- 200 OK : Requête traitée avec succès.
- 201 Created : (non utilisé ici)
- 400 Bad Request : Données manquantes ou mal formatées.
- 404 Not Found : Endpoint inexistant.
- 500 Internal Server Error : Erreur serveur (modèle non chargé, exception im prévue).

3.2 Exercice 2 : Analyse statistique descriptive à la demande

Contexte Un client souhaite obtenir un résumé statistique complet d'une série de nombres sans avoir à ouvrir un notebook.

Objectifs

- Endpoint POST /stats recevant un tableau de valeurs numériques au format JSON
- Calculer avec numpy : moyenne, médiane, écart-type, min, max, quartiles
- Détecter les outliers avec la méthode IQR
- Générer et retourner un histogramme (PNG) ou boxplot
- Permettre de choisir le type de graphique via un paramètre (plot_type=hist|box)

Étapes

1. Récupération du tableau {"data": [1.2, 3.5, ...]}
2. Calculs statistiques avec numpy
3. Génération du graphique avec matplotlib
4. Envoi de l'image en réponse directe ou encodée en base64 dans un JSON avec les statistiques
5. Implémentation d'un paramètre ?format=json pour obtenir les stats seules

Exemple d'utilisation

```
import requests
response = requests . post ( 'http :// localhost :5000/ stats ? plot =box ' , json ={ 'data ' : [10
,20 ,30 ,40 ,100]})
with open ( 'boxplot .png ' , 'wb ' ) as f :
    f . write ( response . content )
```

4 Indications supplémentaires

4.1 Documentation de l'API

Rédiger un fichier README.md expliquant comment installer les dépendances, lancer l'API et utiliser les endpoints avec des exemples curl ou Python requests.

4.2 Gestion des erreurs

- Utiliser des blocs try/except pour capturer les erreurs de format JSON, de clés manquantes, de valeurs aberrantes.
- Retourner des messages d'erreur explicites avec le code HTTP approprié.

4.3 Tests d'intégration

Utiliser Postman, curl et un script Python pour envoyer plusieurs requêtes et vérifier la cohérence des prédictions.

5 Rapport et Livrables

5.1 Livrables attendus

- Rapport écrit (voir section dédiée ci-dessous).
- Code source complet et commenté.
- Fichier requirements.txt avec toutes les dépendances.
- Modèles sauvegardés dans le dossier model/.

5.2 Contenu du rapport

- Introduction et objectifs.
- Description du dataset et analyse exploratoire succincte.

4

- Méthodologie de modélisation (chargement, prétraitement, modèles testés, sélection).
- Résultats de l'évaluation (métriques, matrice de confusion, courbe ROC si pertinent).
- Architecture de l'API (diagramme simple, explication des endpoints). — Extraits de code commentés pour les parties clés.
- Exemples de requêtes/réponses.
- Difficultés rencontrées et perspectives d'amélioration.
- Conclusion.

6 Critères d'Évaluation

Critère	Pondération
Qualité du modèle (justesse des choix, évaluation rigoureuse)	15%
Implémentation correcte de l'API (respect des spécifications, gestion d'erreurs)	35%
Qualité du code (lisibilité, commentaires, structuration)	20%
Documentation et rapport (clarté, exhaustivité)	20%
Tests et validation (présence de tests, exemples fonctionnels)	10%

7 Fichier requirements.txt minimal

```
flask
numpy
matplotlib
scikit - learn
joblib
```

5